

ICC - 15062026 - PROBLEMS

Status	Finished
Started	Monday, 15 June 2026, 2:20 PM
Completed	Monday, 15 June 2026, 3:20 PM
Duration	59 mins 55 secs

Question 1

Complete

Flag question

Construct a deterministic TM of the kind you prefer, which decides the following language:

$$L = \{w \in \{0, 1\}^* \mid w \text{ has 1s at all even positions}\}.$$

As an example, the strings 0111, 010, and ϵ are in the language, while 001 is not in it. Study the time complexity of TM you have defined.

the strings that can be accepted 0111, 010 , empty string
the string need to be rejected 001
alphabets = {start,blank,0,1}
states Q: {qinit,qaccept,qreject,q1,q0}

q1odd= 1 found on odd position
q1even= 1 found on even position
q0 = 0 found on odd position

Transitions:
qinit, start -> qinit,start,R

qinit,0->q0,0,R
qinit,1->q1odd,1,R
qinit, blank -> qaccept,blank,S \\ ϵ can be in the language

q1odd,1->q1even,1,R
q1odd,0 ->qreject,0,R \\reject because 0 is present at even position
q1odd,blank -> qaccept,blank,S \\if there is only 1 in the string accept it

q1even,1->q1odd,1,R
q1even,0 -> q0 , 0, R
q1even,blank -> qaccept,blank,S

q0,1->qeven1,1,R
q0,0-> qreject,0,S
q0,blank -> qaccept,blank,S

the complexity is in O(n) as we scan through the length n of the L and we scan once from left to right.
we reject only when 0 is present in the even position position of the string else we accept

Comment:
TM doesn't recognise the language.

Question 2

Complete

Flag question

The Fundamental Theorem of Arithmetic states that any natural number greater than 1 can be written uniquely as the product of finitely many prime numbers. You are required to prove that the following problem L is in **NP**. To do that, you can give a TM or define some pseudocode. The language L includes precisely those natural numbers which can be written as the product of exactly 2 prime numbers, each of them occurring one or more times in the product. In doing so, you can assume to have a polytime algorithm deciding primality of natural numbers and working in polynomial time. Examples of numbers in L are $14 = 2 \cdot 7$, but also $72 = 2^3 \cdot 3^2$.

certificate: S = p,q \\exactly two prime numbers
the certificate S is the list of exactly two number p and q that can be multiplied and given the natural number n in L
verifier(n,S):
1. if p is not prime:
reject
2. if q is not prime
reject
3.for x in S: \\works til length of S
if xmodn !=0
reject
4.product = 1
for u in S:
product = product x u
if product !=n
reject
5.if n != p**q x q**p \\ we make sure that 72=2**3 x 3**2 conditions satisfies
reject
6.Accept

since we know that primality can be tested in polynomial time so the checking for 1,2 i.e if p and q are prime is in polynomial, for 3 we check till the length of certificate S so its in also polynomial time and the multiplication is also takes polynomial time as we know already and in 5 it is also in polynomial time.

Hence the problem L is in NP because all the instructions have been executed in polynomial time.

Comment:
The algorithm is almost correct, although the exponents are not managed correctly.

Question 3

Complete

Flag question

During the course we considered the problem **KNAPSACK**, which takes as input the weights w_i and the values p_i of n objects, and asks whether there exists a subset of the objects such that the total weight is less than a given threshold z , while the total value is greater than a given threshold k . Consider a variation of the problem, called **VARKNAPSACK**, in which the weights are all equal to 1, and study its computational complexity.

the problem is in P because the weights are equal to 1 so the problem is restricted and since P is a subset of NP , it can be in NP as well but it easier to say its in P
algorithm:
input : list L of p value given of n objects, threshold k is given
output: set of objects whose weight =1 and value less than threhold z
varknapsack()
s = empty
sum = 0
for x in L:
sum = sum plus x
if sum greater than z
return True
if weight is equal to 1
return True
add values into set P \\ so that the n is ready as a subset satisfying certain condition
Return False

since all the instruction works in polynomial time , we conclude that the problem is solvable in polynomial hence in P and we can verify easily as well by given the certificates if the p values and we can verify so its also in NP as well

Comment:
There are some vague ideas, but overall what your algorithm does is not clear. Why z is considered as an output? The output is a truth value.

Finish review